

Freeform Preference Learning for Robotic Manipulation

Marcel Torne* Anubha Mahajan* Abhijnya Bhat* Chelsea Finn

Stanford University

Abstract: Reward design remains a central bottleneck for autonomous robot policy improvement, especially in long-horizon manipulation tasks where sparse success labels provide too little signal and binary preferences collapse many competing notions of quality into one ambiguous signal. We introduce *Freeform Preference Learning (FPL)*, a method for learning robot policies from freeform human preferences. Rather than asking annotators which of two trajectories is better overall, *FPL* lets them define natural-language preference axes, such as speed, safety, quality of placement, or carefulness, and provide pairwise preferences along each axis. These annotations are used to learn a language-conditioned reward model that maps a trajectory and preference label to an axis-specific reward. We use this model to train a reward-conditioned policy that optimizes across the multiple human-specified dimensions. Across four real-world and two simulated long-horizon manipulation tasks, *FPL* improves over sparse-reward and binary-preference methods by 38 percentage points. Beyond improved performance, *FPL* learns dense progress signals without explicit subtask segmentation, shows compositionality of behavior not present in the data, and allows users to steer the policy towards different behaviors at test time without retraining. Blog post with videos available at freeform-pl.github.io/fpl.website/

Keywords: Reinforcement Learning, Preference Learning, Robot Manipulation

1 Introduction

Rewards are a critical component for autonomous robot improvement. An ideal reward function should provide dense, unambiguous feedback and should capture all aspects of desirable behavior. For example, supervision for the simple task of setting a table should incorporate the configuration of the cutlery, the degree of care taken to not break fragile plates, the comfort of nearby people (e.g. to avoid motions that point a knife towards a person), and the speed of execution, among other aspects. Accurately capturing all of these axes presents a major challenge, both when eliciting supervision from people and when representing all of these factors in a reward function and downstream behavior. Moreover, a reward function that captures these axes but is too sparse, or dense but inaccurate, can lead to unwanted downstream behaviors when optimized against. In this paper, we study how to leverage human supervision to learn reward functions and ultimately robot behavior that captures all dimensions of a person’s intent.

Prior works have studied a variety of rewards and reward learning approaches. Perhaps the simplest option is to provide or learn from binary success labels [1, 2, 3], which should in principle make it easy for people to determine if all criteria are met. However, this reward signal places significant burden on the reinforcement learning algorithm, making it hard to scale to more challenging tasks and to incorporate real-world constraints on behavior beyond basic task completion. Other works learn shaped scalar rewards [4, 5, 6] but still focus on task progress metrics, ignoring important criteria on how a task was performed. Finally, preference learning [7, 8, 9] is a promising paradigm

*Equal contribution

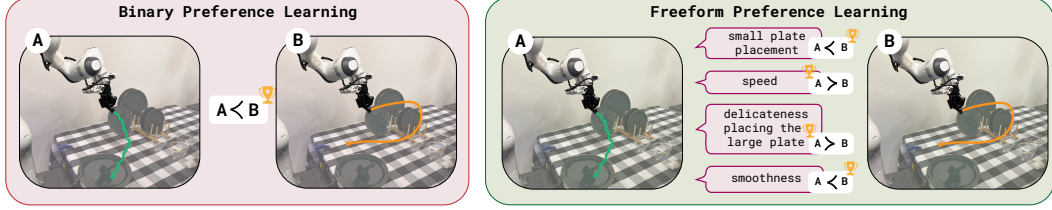


Figure 1: Comparison between binary preference learning (left) and *Freeform Preference Learning* (right). Binary preference learning asks annotators to compare two trajectories using a single “overall preference”. *Freeform Preference Learning* instead collects more detailed feedback by letting the annotator specify the axes in natural-language and provide per-axis preferences.

for learning denser reward signals while reducing the burden on human supervisors, but requires them to collapse multiple axes of judgment into a single “overall” binary preference label. In long-horizon tasks this can make preferences difficult to provide and the resulting supervision ambiguous.

To preserve these axes of judgment, our key insight is to allow annotators to provide preference labels on any task-relevant axes of their choosing. This multidimensional supervision can then be used to learn a policy optimized for the combination of these dimensions. We instantiate this idea by asking annotators to specify relevant judgment dimensions in natural language and to provide a binary preference for each axis. The axes can be defined up front or during the annotation process. We then learn a multi-axis reward function that produces a scalar reward score when conditioned on a natural language description of the axis. Finally, we train a promptable policy to optimize the combination of axes described during reward training. Notably, this framework simultaneously improves both the ease of providing unambiguous supervision and the density of supervision for downstream policy optimization.

The main contribution of our paper is *Freeform Preference Learning (FPL)*, a method for eliciting and learning from freeform human preferences. *FPL* learns a language-conditioned reward function from preferences, capturing a variety of task-relevant attributes including quality of result, speed, smoothness, damage, and hygiene. This reward model provides dense supervision for training a multi-axis reward-conditioned policy. We evaluate *FPL* on four real-world tasks—putting a cube in a target bowl, folding shorts, plating a toast, and setting up a table—as well as two simulated tasks. Across settings, policies trained with *FPL* significantly outperform those trained with sparse rewards and binary preference learning methods. We further find that preserving the multi-dimensionality of feedback enables compositional generalization and test-time steerability of the resulting policies as well as qualitatively denser rewards on long-horizon tasks without requiring subtask segmentation.

2 Preliminaries

In this section, we will give an overview of learning from binary human preferences and, with particular emphasis on the foundational work of Christiano et al. [7].

Reward learning from binary human feedback: Learning from binary human feedback offers a way to provide dense reward signal by learning a reward from human preferences. This avoids the challenge of designing a dense reward based on images. When learning from human feedback, two states or segments (s_i, s_j) from two different trajectories are presented to the annotators. The annotator is then asked to indicate their preference $y \in \{0, 1\}$ between the two, where we define $y = 1$ if the annotator prefers s_i over s_j , and $y = 0$ otherwise. Multiple such pairs are compared and form the dataset of pairwise preferences, $\mathcal{P}$. The Bradley-Terry model assumes that the probability of preferring s_i over s_j , where $\sigma(\cdot)$ is the logistic function, is:

$$P(s_i \succ s_j) = \frac{\exp(r_\phi(s_i))}{\exp(r_\phi(s_i)) + \exp(r_\phi(s_j))} = \sigma(r_\phi(s_i) - r_\phi(s_j)), \quad (1)$$

The reward model r_ϕ is then trained by minimizing the negative log-likelihood of the preferences:

$$\mathcal{L}_{\text{BT}}(\phi) = -\mathbb{E}_{(s_i, s_j, y) \sim \mathcal{P}} [\log \sigma((2y - 1) [r_\phi(s_i) - r_\phi(s_j)])], \quad (2)$$

In practice, providing binary preferences is ambiguous and noisy. In Section 3 we propose a new method to learn from freeform preferences instead.

Policy extraction from the learned reward: After learning the reward model, we must extract the policy that maximizes the reward. Christiano et al. [7] proposed using Proximal Policy Optimization [10], an online on-policy RL algorithm. Using this class of algorithms has proven to be effective for robotics in simulation, as well as for large language models [11] where obtaining rollout data is not costly. In practice, for real-world robot learning, PPO is too sample inefficient. In Section 3, we propose using another class of policy extraction algorithms, off-policy and batch online, that make real-world RL tractable.

3 Learning from Freeform Preferences

The key idea behind our method, *Freeform Preference Learning (FPL)*, is to learn from natural language and open-ended feedback instead of traditional binary preferences. Rather than asking annotators for one overall preference between two trajectories, we ask them to describe the axes along which to compare them, such as speed, safety, smoothness, or subtask completion. This yields feedback that is more granular and less ambiguous. We use these freeform preferences to learn a language-conditioned, multi-dimensional reward function that scores trajectories along each specified axis. We then train a policy conditioned on multiple preference dimensions to optimize the behavior with respect to each axis. We describe each component of our proposed algorithm below.

3.1 Learning a Reward Function from Freeform Human Feedback

Traditionally, human preferences are collected as a binary signal over an “overall quality” metric, i.e., the annotators are asked “which one of the two trajectories do you prefer?” However, this signal is difficult to provide because the answer often depends on the axis of comparison. For example, if one trajectory is faster while the other is safer, it is unclear which should be preferred overall. This ambiguity is further amplified when preferences are collected over trajectory segments, since the two segments may correspond to different stages of the task and therefore be difficult to directly compare. We instead collect freeform human preferences by showing two full trajectories and asking them to evaluate them along multiple axes, either predefined or specified by the annotator in natural language. Rather than asking a single fixed question about “overall quality”, we collect preferences over a variety of axes such as “formality of setup”, “speed”, “safety”, and so on. Given a freeform preference dataset $\mathcal{P}$ made up of natural language labels defining the axes l_k and binary preferences y_k per axis, we now describe how to learn a reward function r_ϕ from it.

A simple way to learn from multi-dimensional feedback would be to define a fixed set of K preference axes and train a separate reward function for each one [12]. However, this requires a predefined set of axes and limits generalization across semantically similar descriptions. For example, different annotators may refer to the same concept as “speed”, “fast”, or “efficient”. We instead keep preference axes in natural language and condition a single reward model directly on their text descriptions, leveraging the pretrained representations of vision-language models, see Fig. 2.

The Bradley-Terry model in Eq. 2 is typically used to learn a uni-dimensional reward [7, 13, 14]. We extend this formulation to freeform preferences by conditioning the reward model on the natural-language axis for which each preference was provided. This yields an axis-conditioned reward function r_ϕ that scores a trajectory with respect to the specified axis, as defined in Eq 3.

For a pair of trajectories (τ_i, τ_j) , an annotator provides per-axis preference labels and language labels $\{(l_k, y_k) \mid k = 1, \dots, K_{ij}\}$, where $y_k \in \{0, 1\}$ indicates the preferred trajectory along axis k and K_{ij} is the total number of preference axes described for the trajectory pair (which may vary across pairs). The multi-dimensional reward model r_ϕ outputs a single scalar conditioned on the axis label and is trained to minimize the Bradley-Terry negative log-likelihood:

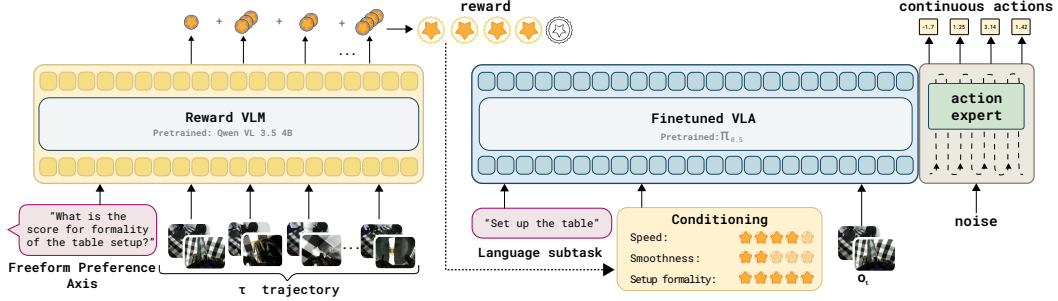


Figure 2: *FPL* learns a multi-dimensional reward function to score the complete trajectory τ conditioned on the natural language preference axis. To leverage the multi-dimensionality of the reward function, *FPL* learns to reproduce behavior conditioned on the reward over multi-dimensional axes in text form. At test-time we can steer it towards high reward behaviors.

$$\mathcal{L}_{FPL-Reward}(\phi) = -\mathbb{E}_{(\tau_i, \tau_j, \{(l_k, y_k) | k=1, \dots, K_{ij}\}) \sim \mathcal{P}} \left[\sum_{k=1}^{K_{ij}} \log \sigma \left((2y_k - 1) (r_\phi(\tau_i | l_k) - r_\phi(\tau_j | l_k)) \right) \right], \quad (3)$$

Finally, unlike standard preference-based reward models that score individual observations or short segments [7], we condition the reward model on the observation history. This allows the model to capture temporal dependencies that may be necessary for evaluating trajectory-level preferences. Given a trajectory $\tau = o_1, \dots, o_T$ and a natural-language preference axis l_k , we define the trajectory-level reward as a sum of per-prefix scores:

$$r_\phi(\tau, l_k) = \sum_{i=1}^T g_\phi(o_{1:i}, l_k), \quad (4)$$

where g_ϕ is a multimodal transformer that scores the segment $o_{1:i}$ conditioned on the axis l_k .

3.2 Policy Extraction on a Multi-Dimensional Reward Function

Now that we have described how to learn a language-conditioned reward function from freeform feedback on trajectory pairs, we discuss how best to use this supervision to learn a policy. One option would be to collapse the different dimensions into a scalar reward by a weighted sum [15] and optimize it with standard RL techniques. Although this is simple and would leverage the benefits of easier-to-provide feedback, policy optimization can also benefit significantly from more detailed supervision. Collapsing all the axes into a single scalar is more prone to reward hacking and exhibiting the problems of reward shaping [16]. Moreover, as we show in Section 5.2, preserving the decomposed rewards enables the learned policy to exhibit compositionality of behaviors not present in the original dataset, as well as at test-time steerability without retraining with a different reward. To realize these benefits, we train a policy conditioned on natural-language reward axes and their corresponding scores.

In principle, there are many base RL algorithms that we can extend to the language-conditioned setting with a language-conditioned reward function. We opt to use a particularly simple approach, based on prior work that trains reward-conditioned policies [17, 18, 19, 20]. Because we would like our policy to be able to optimize all representative axes of preferences rather than just one, we select a comprehensive set of K_π preference axes for policy training $L = \{l_k | k = 1 \dots K_\pi\}$. These can be selected as representative axis descriptions that appear in the reward model training dataset, either manually or through automatic summarization to remove synonym phrases. Then, we condition the policy on all of the axis descriptions l_k and corresponding trajectory rewards $r_\phi(\tau | l_k)$. More formally, our policy training objective is:

$$\mathcal{L}_{FPL-Policy}(\theta) = -\mathbb{E}_{\tau_i \sim \mathcal{D}} \left[\sum_{t=1}^{T_i} \log \pi_\theta(\mathbf{a}_i^t | s_i^t, l_1, r_\phi(\tau_i | l_1) \dots l_{K_\pi}, r_\phi(\tau_i | l_{K_\pi})) \right], \quad (5)$$

This approach can be used in both an offline and online RL setting. In the former case, this policy optimization step is performed once on an offline preference dataset. In the latter case, we repeat this process: collecting roll-out data from the latest policy, soliciting preference annotations on this new data, and then updating the policy.

Notably, the preference axes described by human annotators may evolve over multiple iterations as the policy becomes more capable, and likewise, the set of K_π preference axes for policy optimization can also change and expand. This naturally can yield a curriculum where initial preferences focus on initial stages of the task or coarse attributes of behavior, while later preferences can focus on later stages of the task and fine details. The full iterative training process is outlined in Algorithm 1.

At test-time, *FPL* can steer the policy towards high-performing behaviors by varying the target rewards used for conditioning. Because the reward model outputs are unbounded, selecting values that both elicit the desired behavior and remain in distribution can be difficult. We therefore standardize rewards per axis over $\mathcal{D}$, yielding a normalized scale for more easily querying the policy at test time.

Algorithm 1 *FPL: Freeform Preference Learning*

Require: Initial offline dataset $\mathcal{D}$, number of iterations N , π_0 initial policy
 $\mathcal{P} \leftarrow \emptyset$; Initialize preference buffer
for $n = 1, \dots, N$ **do**
 $\mathcal{P} \leftarrow \mathcal{P} \cup \mathcal{P}_n$; Collect $\mathcal{P}_n$ freeform preferences over pairs from $\mathcal{D}$
 $r_\phi^n \leftarrow$ Train reward model on $\mathcal{P}$ (Eq. 3)
 $L_n \leftarrow$ Extract preference axes from $\mathcal{P}$
 $\pi_n \leftarrow$ Train π_n using r_ϕ^n on $\mathcal{D}$ (see Eq 5)
 if $n \neq N$ **then**
 $\mathcal{T}_n \leftarrow$ Roll out π_n
 $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{T}_n$
return final policy π_N

4 Related Work

Real-world reinforcement learning on vision language action models: Performing reinforcement learning in the real world on vision language action models (VLAs) [21, 20, 22, 23, 24] is challenging for several reasons. VLAs typically use a generative action head [21, 25], and a line of prior work studies how to apply RL to such generative policies [26, 27, 28]. To make real-world RL more sample-efficient, we build on reward-conditioning [19, 17], an off-policy policy-extraction approach that can also be applied offline [19, 29, 20, 17, 18]. However, all prior work conditions on a single scalar return. In particular, Intelligence et al. [20] proposes to learn a value function from success/failure and time-to-go signals and conditions the VLA on a binarized advantage. While effective, this signal is sparse, derived from a single task outcome, which limits its effectiveness on long-horizon tasks where successes might be scarce. Other works instead derive the value function from pretrained language models [4, 6], but likewise reduce it to a single scalar. We instead learn the reward model and condition the policy on multiple reward axes simultaneously, leading to denser signal and policy capabilities such as compositionality of behaviors and steerability. Relatedly, multi-objective RL learns policies conditioned on a reward-weight vector over a fixed set of objectives [15, 30, 31, 32], allowing behaviors to be steered at test time. Unlike these methods, we do not assume predefined axes or rewards, but learn from freeform human preferences.

Reinforcement learning from human preferences: Learning from human preferences sidesteps the difficulty of hand-designing reward functions, and has proven effective for large language models [11, 33, 34]. It is especially valuable in real-world robotics from image observations, where engineering a reward function is notoriously hard. In robot learning, Christiano et al. [7] and much subsequent work [35, 14, 36, 37, 38, 39] learn a reward model from binary preferences comparing two robot segments. However, a single overall preference on a long-horizon task is often ambiguous and we instead collect freeform preferences in which annotators specify the axes they wish to judge to reduce the ambiguity. A complementary line of work studies which comparisons to query, using active selection to learn reward functions from fewer preferences [9, 40]. Bobu et al. [41] decompose reward learning into explicitly taught reward features [42, 43] that are combined into a single reward

and Agnihotri et al. [44] optimizes a primary objective while enforcing lower bounds on secondary objectives. In contrast, we let annotators specify preference axes directly in natural language and optimize the policy over these. A separate direction leverages the state-of-the-art VLMs to obtain binary preferences [45], some querying for specific substeps to obtain denser supervision [8]. However, these do not leverage the granularity and multi-dimensionality of rewards as we propose with *FPL*.

Learning from natural language supervision: Denser human feedback aims to provide a stronger learning signal to the RL process. In the LLM setting, Wu et al. [12] trains multiple reward models, each tied to a distinct error category. In contrast, we learn a single language-conditioned reward model from freeform human-specified axes rather than a fixed, predefined category set. Other methods leverage richer feedback through the model’s capacity for in-context adaptation from natural language, for example rich human feedback for text-to-image generation [46] and open-ended text optimization via pairwise comparison [47]. While in-context adaptation is appealing, current VLAs cannot react to language feedback in-context in the same way making such methods inapplicable to robot learning. Prior robotics work has used natural language to correct robot plans [48], and to infer preference-conditioned state abstractions that identify task-relevant features for reward inference [49]. In contrast, we use language not merely as corrective feedback, but as an explicit interface for defining the axes along which preferences are elicited and rewards are learned. Finally, Hirota et al. [50] learn a shared latent space between comparative language and trajectories, and Hwang et al. [51] treat freeform rationales attached to preferences as projection axes in an embedding space. Both must first learn a dedicated space in which language and trajectories are aligned, and both use that language to sharpen a single scalar reward, whereas we learn an open-ended language conditioned reward model and train the policy with unmerged axes, letting it compose behaviors absent from the demonstration data.

5 Analysis & Experimental Evaluation

Our experimental section is designed to answer the following questions: (a) Does *FPL* learn effective policies through freeform human preferences? (b) Do policies learned through *FPL* exhibit compositionality of behaviors unseen in the data? (c) Does *FPL* exhibit steerability of rewards at test time? (d) Does *FPL* learn denser reward functions for long-horizon tasks?

5.1 Experimental Setup

In order to empirically respond to the questions above, we consider four real-world manipulation tasks and two simulation tasks.

Real-world. *Put cube into target bowl* is a diagnostic task for steerability, where the policy is reward-conditioned to place the cube into one of three bowls. *Fold shorts* tests deformable-object manipulation, requiring the robot to fold shorts in three folds while optimizing speed and alignment. *Plate toast* tests dexterous tool use, requiring the robot to transfer toast from a tray to a plate while using the spatula smoothly. *Set up the table* is a long-horizon task in which the robot places two plates, cutlery, and a cup, testing task completion, formality, and carefulness. All real-world tasks use the DROID setup [52], with two camera views as observations and joint-velocity control for a Franka robot. The tasks are shown in Figure 7; further details are in Appendix 7.1.

Simulation. We use two Robomimic-based tasks [53]. *Object rearrangement* requires placing two objects into the correct containers and in the correct order. *Bimodal square* tests compositionality, the target behavior is to place the nut on the right peg quickly, while the initial dataset contains fast left-peg trajectories and slow right-peg trajectories. We also evaluate *bimodal square (inverted)* to test whether the same policy can be steered at test time to place the nut on the left peg. More details can be found in Appendix 7.2.

All tasks start from offline demonstrations with varying quality and strategies. We report mean and standard error, using 20 rollouts per real-world method and three seeds in simulation (see Apdx. 7).

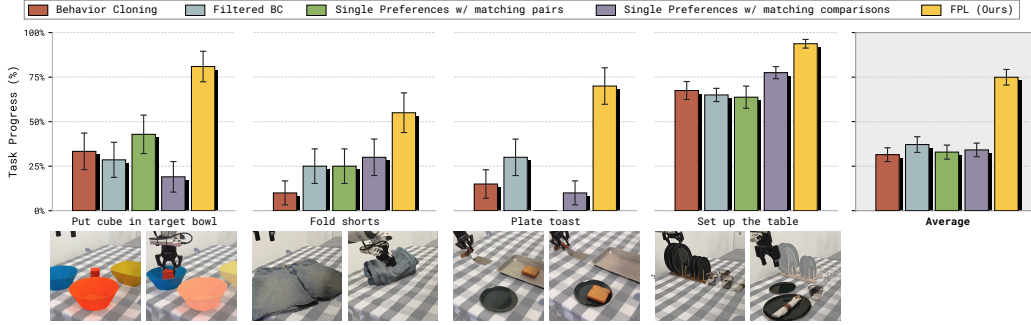


Figure 3: Baseline performance on real-world tasks. *FPL* learns more performant policies across all tasks, with an average improvement of 38 percentage points over the second-best baseline. The signal from binary preferences is too ambiguous to learn performant policies. The sparse success signal is too weak to provide the necessary supervision to solve the tasks successfully.

Table 1: Simulation results comparing *FPL* against baselines across simulation environments. Multi-dimensional preferences as leveraged by *FPL* provide the best supervision. *FPL* can successfully solve with the same policy the *bimodal square* and *bimodal square inverted* benchmarks by changing the reward conditioning at test-time.

Method	Object rearrangement (success)	Bimodal square (throughput)	Bimodal square inverted (throughput)
BC [55]	0.04 ± 0.02	0.71 ± 0.07	0.25 ± 0.14
Filtered BC [29]	0.09 ± 0.05	0.71 ± 0.02	0.00 ± 0.00
Weighted Regression [54]	0.18 ± 0.08	0.83 ± 0.08	0.09 ± 0.06
Advantage Conditioning [20]	0.17 ± 0.06	0.75 ± 0.12	0.08 ± 0.04
Single Preferences (match pairs) [7]	0.73 ± 0.04	0.67 ± 0.08	0.78 ± 0.01
Single Preferences (match comparisons)	0.79 ± 0.06	0.73 ± 0.05	0.74 ± 0.05
<i>FPL</i> (Ours)	0.84 ± 0.09	1.19 ± 0.01	1.24 ± 0.01

5.2 Experimental Evaluation

Comparisons. We compare *FPL* against five baselines. *Single Preferences* learns from standard pairwise preferences over a single “overall quality” axis [7], using the same reward-conditioned policy extraction as *FPL* for a fair comparison. *Advantage Conditioning* following Intelligence et al. [20], we train a value function from success signal and time-to-go supervision, and condition the policy on the resulting advantage. *Weighted Regression* uses the multi-dimensional reward model learned by *FPL*, but extracts the policy with weighted regression using the average of rewards across the axes L [54]. *Filtered BC* trains on the offline dataset together with successful policy rollouts, providing a sparse reward policy-extraction baseline [29]. *BC* trains with imitation learning over the original offline dataset without reward learning or iterative improvement [55].

***FPL* learns performant policies from freeform human preferences.** In Figure 3, we show that *FPL* outperforms all baselines in the real-world, improving by 38 overall percentage points over the next best method. We find that single overall preferences are often ambiguous in long-horizon tasks. For example, in *plate toast*, one trajectory may use the gripper fingers instead of the spatula, making it unhygienic, while another may use the spatula but drop the toast. In such cases deciding which trajectory is “better overall” is difficult and leads to noisy supervision. Sparse success/failure rewards are also insufficient: in *setup table*, the robot must complete several sequential subtasks, so rewarding only perfect executions provides too little signal, while rewarding imperfect executions can reinforce undesirable behavior. In practice, sparse-reward baselines learned to place the items in the correct locations but did not learn to place the items carefully and often dropped the plates instead of placing them with care. In contrast, *FPL* provides axis-specific supervision, allowing the policy to improve both the task completion and qualitative aspects of behavior.

In simulation, *FPL* outperforms all baselines, as shown in Table 1. The gap is particularly clear in the *object rearrangement* task where the long-horizon and noisiness of the offline dataset has few complete successes. In this setting, success/failure signal is too sparse to correctly learn a successful policy. By contrast, preference-based methods provide denser signal. But, single overall preferences

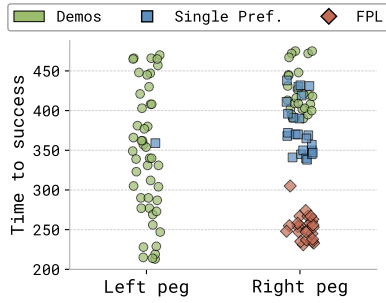


Figure 4: Time to success per peg and per baseline. *FPL* achieves fast trajectories on the right peg despite the demo data not including fast data on the right peg, only on the left peg. *FPL* exhibits compositionality of behaviors while baselines do not.

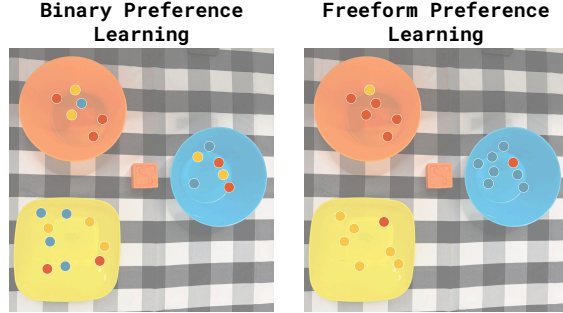


Figure 5: Each dot represents the block’s final position, with the color corresponding to the target bowl’s color. With *FPL* we can steer the policy at test-time by prompting a different reward to optimize while the baselines cannot.

still collapse multiple behaviors into one ambiguous signal. As a result the policies learned from single preferences achieve the correct arrangement of objects but fail since they often drop objects from too high above the target. *FPL*, however, uses multi-dimensional preferences to capture the different important axes for the task separately, leading to successful policies.

***FPL* exhibits compositionality of behaviors.** We study this in the *bimodal square* simulation environment, where the goal is to place the nut on the right peg. As shown in Figure 4, the offline dataset has both fast and slow demonstrations for the left peg but *only* slow demonstrations for the right peg. *FPL* achieves faster right peg placements than those seen in the training data, while the single preference baseline does not improve beyond the demonstrated behaviors. This shows that *FPL* can compose behaviors through the preference axes, in this case the target placement at faster speed, whereas the baselines cannot (see Table 6). We attribute this compositionality to multi-dimensional reward learning together with reward-conditioned policy extraction.

***FPL* exhibits test-time steerability.** We evaluate steerability using the *inverted* version of *bimodal square*, where the same trained policy is conditioned to place the nut on the left peg instead of the right peg. As we show in the *inverted* columns of Table 1 and Figure 5, *FPL* is the only method that achieves high performance on both the original and the inverted tasks with the same policy. This is enabled by reward-conditioned policy extraction on multi-dimensional rewards: because *FPL* trains on trajectories across the replay buffer without filtering, the policy observes both high- and low-scoring behaviors along each axis and can be steered at test time by changing the target reward conditioning.

5.3 Qualitative Analysis

***FPL* qualitatively produces denser reward signals on long-horizon tasks without explicit task segmentation.** In Figure 6, we qualitatively compare reward models learned with *FPL* and binary preference feedback on an example rollout from the *setup table* task. Although neither model is trained with explicit subtask boundaries, the reward learned with *FPL* temporally localized the corresponding events such as placing the big plate, small plate, cup, and cutlery. This makes the learned reward more interpretable and suggests that freeform, axis-specific preferences can provide a denser signal for long-horizon tasks. In contrast, the binary-preference reward produces a large reward spike near the end of the episode, despite no major subtask being completed at that point showing then an error in the credit assignment.

Freeform preferences axes naturally change throughout iterations. With the policy performance improving at each iteration, the annotators change from more coarse feedback to more concrete and perfectionist feedback. In Figure 10, we observe that in the early iterations of *fold shorts* task,

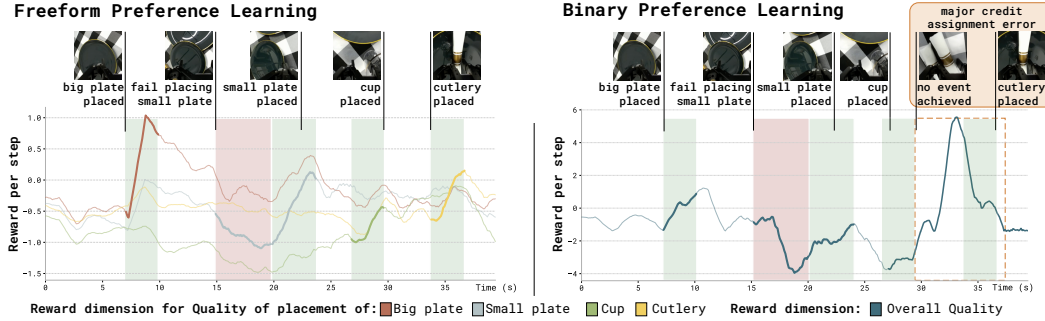


Figure 6: Qualitative reward analysis on *setup table*. We compare rewards learned from freeform preferences with *FPL* (left) and from binary preferences (right) on a representative rollout. For visualization, we show a subset of *FPL*’s reward axes. The axis-conditioned rewards learned by *FPL* are temporally localized around their corresponding subtask events, while the binary-preference reward collapses behavior into a single score and produces a large spike near the end of the trajectory despite no major subtask being completed.

the annotator focuses on whether each fold happens at all, and later ones, once folding is reliable, whether there are wrinkles and final alignment are perfected.

Freeform preferences produced diverse preference axes. Allowing annotators to specify preference axes in natural language yields a diverse set of labels. In the *plate toast* task, we obtained 295 trajectory-pair comparisons and over 1477 axis-level comparisons spanning 41 distinct labels (see Figure 14 in Appendix 8). This suggests the annotators naturally use a wide range of criteria when evaluating robot behavior, motivating the need for preference learning methods that preserve this structure rather than collapse feedback into a single overall score.

Freeform preferences reduce annotation time per label. As shown in Figure 9b, collecting freeform preferences is approximately 50% faster per label than collecting single binary preferences. Because annotators provide multiple axis-specific judgments for each trajectory pair, the cost of viewing the videos is shared across several labels, which reduces the annotation overhead.

6 Limitations & Conclusion

To conclude, we introduced *FPL*, a method for learning robot policies from freeform human preferences. By collecting preferences with natural-language axes, *FPL* provides denser and less ambiguous supervision than single binary preferences. Across four real-world tasks and two simulation settings, *FPL* outperforms the baselines. We show that preserving the multi-dimensional structure of human feedback enables compositionality of behaviors and test-time steerability of the learned policy as well as qualitatively learn reward models with better credit assignment.

Several limitations remain. Preference learning requires collecting human preferences, which is more expensive than fully unsupervised approaches. Reward-conditioned policy learning requires selecting appropriate reward values at test time, automating this selection is an important direction for future work. Finally, the current policy is conditioned on a fixed set of preference axes, and extending this method to handle variable axes is a promising direction as VLAs become more capable.

Acknowledgments

We thank Lars Ankile and Moo Jin Kim for their support with the real world experiments. We thank Alex Swerdlow, Aneesh Muppidi, Yuejiang Liu and the members of the Stanford IRIS lab for their insightful discussions and feedback on the paper drafts. This work was supported in part by the ONR grant N00014-22-1-2621, the Robotics and AI Institute and, the Stanford Institute for Human-Centered AI. C. Finn is a CIFAR fellow.

Contributions

The project was started and designed by MT and CF. MT designed the overall algorithm. MT implemented the reward model and conditioning of the policy. MT designed and implemented the simulation benchmarks and baselines. AM, AB and MT worked on the real world experiments including data and preference collection as well as fine-tuning of the real world policies. CF provided advice throughout the project. MT and CF worked on writing, illustrations, blog post and the video.

References

- [1] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine. Serl: A software suite for sample-efficient robotic reinforcement learning, 2024.
- [2] A. Singh, L. Yang, K. Hartikainen, C. Finn, and S. Levine. End-to-end robotic reinforcement learning without reward engineering. *arXiv preprint arXiv:1904.07854*, 2019.
- [3] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke, et al. Qt-opt: Scalable deep reinforcement learning for vision-based robotic manipulation. In *CoRL*, pages 651–673, 2018.
- [4] A. Liang, Y. Korkmaz, J. Zhang, M. Hwang, A. Anwar, S. Kaushik, A. Shah, A. S. Huang, L. Zettlemoyer, D. Fox, et al. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. *arXiv preprint arXiv:2603.02115*, 2026.
- [5] T. Lee, A. Wagenmaker, K. Pertsch, P. Liang, S. Levine, and C. Finn. Roboreward: General-purpose vision-language reward models for robotics. *arXiv preprint arXiv:2601.00675*, 2026.
- [6] S. Chen, C. Harrison, Y.-C. Lee, A. J. Yang, Z. Ren, L. J. Ratliff, J. Duan, D. Fox, and R. Krishna. Topreward: Token probabilities as hidden zero-shot rewards for robotics. *arXiv preprint arXiv:2602.19313*, 2026.
- [7] P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences, 2017. URL <https://arxiv.org/abs/1706.03741>.
- [8] Z. Zhang, K. Zheng, Z. Chen, J. Jang, Y. Li, S. Han, C. Wang, M. Ding, D. Fox, and H. Yao. Grape: Generalizing robot policy via preference alignment. *arXiv preprint arXiv:2411.19309*, 2024.
- [9] D. Sadigh, A. D. Dragan, S. S. Sastry, and S. A. Seshia. Active preference-based learning of reward functions. In *Robotics: Science and Systems*, 2017. URL <https://api.semanticscholar.org/CorpusID:12226563>.
- [10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017. URL <http://arxiv.org/abs/1707.06347>.
- [11] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback, 2022.

- [12] Z. Wu, Y. Hu, W. Shi, N. Dziri, A. Suhr, P. Ammanabrolu, N. A. Smith, M. Ostendorf, and H. Hajishirzi. Fine-grained human feedback gives better rewards for language model training. *Advances in Neural Information Processing Systems*, 36:59008–59033, 2023.
- [13] K. Lee, L. Smith, and P. Abbeel. Pebble: Feedback-efficient interactive reinforcement learning via relabeling experience and unsupervised pre-training. In *International Conference on Machine Learning*, 2021.
- [14] M. Torne, M. Balsells, Z. Wang, S. Desai, T. Chen, P. Agrawal, and A. Gupta. Breadcrumbs to the goal: goal-conditioned exploration from human-in-the-loop feedback. In *Advances in Neural Information Processing Systems*, 2023.
- [15] M. Hwang, L. Weihs, C. Park, K. Lee, A. Kembhavi, and K. Ehsani. Promptable behaviors: Personalizing multi-objective rewards from human preferences. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16216–16226, 2024.
- [16] T. Chen, M. Tippur, S. Wu, V. Kumar, E. Adelson, and P. Agrawal. Visual dexterity: In-hand reorientation of novel and complex object shapes. *Science Robotics*, 8(84):eadc9244, 2023. doi:10.1126/scirobotics.adc9244. URL <https://www.science.org/doi/abs/10.1126/scirobotics.adc9244>.
- [17] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch. Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems*, 34:15084–15097, 2021.
- [18] J. Schmidhuber. Reinforcement learning upside down: Don’t predict rewards—just map them to actions. *arXiv preprint arXiv:1912.02875*, 2019.
- [19] A. Kumar, X. B. Peng, and S. Levine. Reward-conditioned policies. *arXiv preprint arXiv:1912.13465*, 2019.
- [20] P. Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, et al. $\pi_{0,6}^*$: a vla that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.
- [21] P. Intelligence. π_0 : A vision-language-action flow model for general robot control, 2024. URL <https://arxiv.org/abs/2410.24164>.
- [22] NVIDIA. Gr00t n1: An open foundation model for generalist humanoid robots, 2025. URL <https://arxiv.org/abs/2503.14734>.
- [23] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [24] M. Torne, K. Pertsch, H. Walke, K. Vedder, S. Nair, B. Ichter, A. Z. Ren, H. Wang, J. Tang, K. Stachowicz, et al. Mem: Multi-scale embodied memory for vision language action models. *arXiv preprint arXiv:2603.03596*, 2026.
- [25] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023.
- [26] A. Ren, J. Lidard, L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz. Diffusion policy policy optimization. In *International Conference on Learning Representations*, volume 2025, pages 77288–77329, 2025.

- [27] C. Xu, J. T. Springenberg, M. Equi, A. Amin, A. Esmail, S. Levine, and L. Ke. Rl token: Bootstrapping online rl with vision-language-action models. *arXiv preprint arXiv:2604.23073*, 2026.
- [28] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal. From imitation to refinement-residual rl for precise assembly. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 01–08. IEEE, 2025.
- [29] S. Emmons, B. Eysenbach, I. Kostrikov, and S. Levine. Rvs: What is essential for offline rl via supervised learning? *arXiv preprint arXiv:2112.10751*, 2021.
- [30] M. Peschl, A. Zgonnikov, F. A. Oliehoek, and L. C. Siebert. Moral: Aligning ai with human norms through multi-objective reinforced active learning. *arXiv preprint arXiv:2201.00012*, 2021.
- [31] A. Pan, W. Xu, L. Wang, and H. Ren. Additional planning with multiple objectives for reinforcement learning. *Know.-Based Syst.*, 193(C), Apr. 2020. ISSN 0950-7051. doi:10.1016/j.knosys.2019.105392. URL <https://doi.org/10.1016/j.knosys.2019.105392>.
- [32] K. V. Moffaert, M. M. Dragan, and A. Nowé. Scalarized multi-objective reinforcement learning: Novel design techniques. *2013 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL)*, pages 191–199, 2013. URL <https://api.semanticscholar.org/CorpusID:15980735>.
- [33] D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, and G. Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.
- [34] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- [35] K. Lee, L. M. Smith, and P. Abbeel. PEBBLE: feedback-efficient interactive reinforcement learning via relabeling experience and unsupervised pre-training. In M. Meila and T. Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pages 6152–6163. PMLR, 2021. URL <http://proceedings.mlr.press/v139/lee21i.html>.
- [36] M. Balsells, M. Torne, Z. Wang, S. Desai, P. Agrawal, and A. Gupta. Autonomous robotic reinforcement learning with asynchronous human feedback. *arXiv preprint arXiv:2310.20608*, 2023.
- [37] J. Hejna, R. Rafailov, H. Sikchi, C. Finn, S. Niekum, W. B. Knox, and D. Sadigh. Contrastive preference learning: Learning from human feedback without reinforcement learning. In *International Conference on Learning Representations*, volume 2024, pages 18770–18798, 2024.
- [38] J. Hejna and D. Sadigh. Inverse preference learning: Preference-based rl without a reward function. *Advances in Neural Information Processing Systems*, 36:18806–18827, 2023.
- [39] T. Lin, S. Deng, Z.-H. Yin, P. Abbeel, and J. Malik. How to peel with a knife: Aligning fine-grained manipulation with human preference. *arXiv preprint arXiv:2603.03280*, 2026.
- [40] E. Biyik and D. Sadigh. Batch active preference-based learning of reward functions. In *2nd Annual Conference on Robot Learning, CoRL 2018, Zürich, Switzerland, 29-31 October 2018, Proceedings*, volume 87 of *Proceedings of Machine Learning Research*, pages 519–528. PMLR, 2018. URL <http://proceedings.mlr.press/v87/biyik18a.html>.

- [41] A. Bobu, M. Wiggert, C. Tomlin, and A. D. Dragan. Inducing structure in reward learning by learning features. *The International Journal of Robotics Research*, 41(5):497–518, 2022.
- [42] C. Basu, M. Singhal, and A. D. Dragan. Learning from richer human guidance: Augmenting comparison-based learning with feature queries. In *Proceedings of the 2018 ACM/IEEE International Conference on Human-Robot Interaction*, pages 132–140, 2018.
- [43] A. Peng, Y. Sun, T. Shu, and D. Abel. Pragmatic feature preferences: Learning reward-relevant preferences from human input. *arXiv preprint arXiv:2405.14769*, 2024.
- [44] A. Agnihotri, R. Jain, D. Ramachandran, and Z. Wen. Multi-objective preference optimization: Improving human alignment of generative models. *arXiv preprint arXiv:2505.10892*, 2025.
- [45] Y. Wang, Z. Sun, J. Zhang, Z. Xian, E. Biyik, D. Held, and Z. Erickson. RL-vlm-f: Reinforcement learning from vision language foundation model feedback. *arXiv preprint arXiv:2402.03681*, 2024.
- [46] Y. Liang, J. He, G. Li, P. Li, A. Klimovskiy, N. Carolan, J. Sun, J. Pont-Tuset, S. Young, F. Yang, et al. Rich human feedback for text-to-image generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19401–19411, 2024.
- [47] Y. Lee, J. Boen, and C. Finn. Feedback descent: Open-ended text optimization via pairwise comparison. *arXiv preprint arXiv:2511.07919*, 2025.
- [48] P. Sharma, B. Sundaralingam, V. Blukis, C. Paxton, T. Hermans, A. Torralba, J. Andreas, and D. Fox. Correcting robot plans with natural language feedback. *arXiv preprint arXiv:2204.05186*, 2022.
- [49] A. Peng, A. Bobu, B. Z. Li, T. R. Sumers, I. Sucholutsky, N. Kumar, T. L. Griffiths, and J. A. Shah. Preference-conditioned language-guided abstraction. In *Proceedings of the 2024 ACM/IEEE International Conference on Human-Robot Interaction*, pages 572–581, 2024.
- [50] E. Hirota, Z. Yang, A. Hiranaka, M. Jun, J. Tien, S. J. Russell, A. Dragan, and E. Biyik. Active reward learning and iterative trajectory improvement from comparative language feedback. *The International Journal of Robotics Research*, 2025. doi:10.1177/02783649251394742. Advance online publication.
- [51] M. Hwang, Y. Korkmaz, D. Seita, and E. Biyik. Causally robust reward learning from reason-augmented preference feedback. *arXiv preprint arXiv:2603.04861*, 2026.
- [52] D. Team. DROID: A large-scale in-the-wild robot manipulation dataset. In *Robotics: Science and Systems XX, Delft, The Netherlands, July 15-19, 2024*, 2024. doi:10.15607/RSS.2024.XX.120. URL <https://doi.org/10.15607/RSS.2024.XX.120>.
- [53] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *arXiv preprint arXiv:2108.03298*, 2021.
- [54] X. B. Peng, A. Kumar, G. Zhang, and S. Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019.
- [55] S. Levine, C. Finn, T. Darrell, and P. Abbeel. End-to-end training of deep visuomotor policies. *The Journal of Machine Learning Research*, 17(1):1334–1373, 2016.
- [56] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, P. D. Fagan, J. Hejna, M. Itkina, M. Lepert, Y. J. Ma, P. T. Miller, J. Wu, S. Belkhale, S. Dass, H. Ha, A. Jain, A. Lee, Y. Lee, M. Memmel, S. Park, I. Radosavovic, K. Wang, A. Zhan, K. Black, C. Chi, K. B. Hatch, S. Lin, J. Lu, J. Mercat, A. Rehman, P. R. Sanketi, A. Sharma, C. Simpson, Q. Vuong, H. R. Walke, B. Wulfe,

- T. Xiao, J. H. Yang, A. Yavary, T. Z. Zhao, C. Agia, R. Baijal, M. G. Castro, D. Chen, Q. Chen, T. Chung, J. Drake, E. P. Foster, J. Gao, D. A. Herrera, M. Heo, K. Hsu, J. Hu, D. Jackson, C. Le, Y. Li, K. Lin, R. Lin, Z. Ma, A. Maddukuri, S. Mirchandani, D. Morton, T. Nguyen, A. O'Neill, R. Scalise, D. Seale, V. Son, S. Tian, E. Tran, A. E. Wang, Y. Wu, A. Xie, J. Yang, P. Yin, Y. Zhang, O. Bastani, G. Berseth, J. Bohg, K. Goldberg, A. Gupta, A. Gupta, D. Jayaraman, J. J. Lim, J. Malik, R. Martín-Martín, S. Ramamoorthy, D. Sadigh, S. Song, J. Wu, M. C. Yip, Y. Zhu, T. Kollar, S. Levine, and C. Finn. Droid: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024.
- [57] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, A. Maddukuri, S. Nasiriany, and Y. Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020.
- [58] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- [59] P. Intelligence. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025. URL <https://arxiv.org/abs/2504.16054>.

Appendix

7 Experimental details

In this section, we will go over the details of the simulation 7.2 and real-world 7.1 experiments presented in the paper.

7.1 Real-World Tasks

All four real-world tasks (see Fig. 7) use the DROID setup [56] on a Franka robot, with two camera views (wrist and third person) as observations and joint-velocity control. Each task starts from offline demonstrations of varying quality and strategy. The per-task descriptions and preference axes are described below.

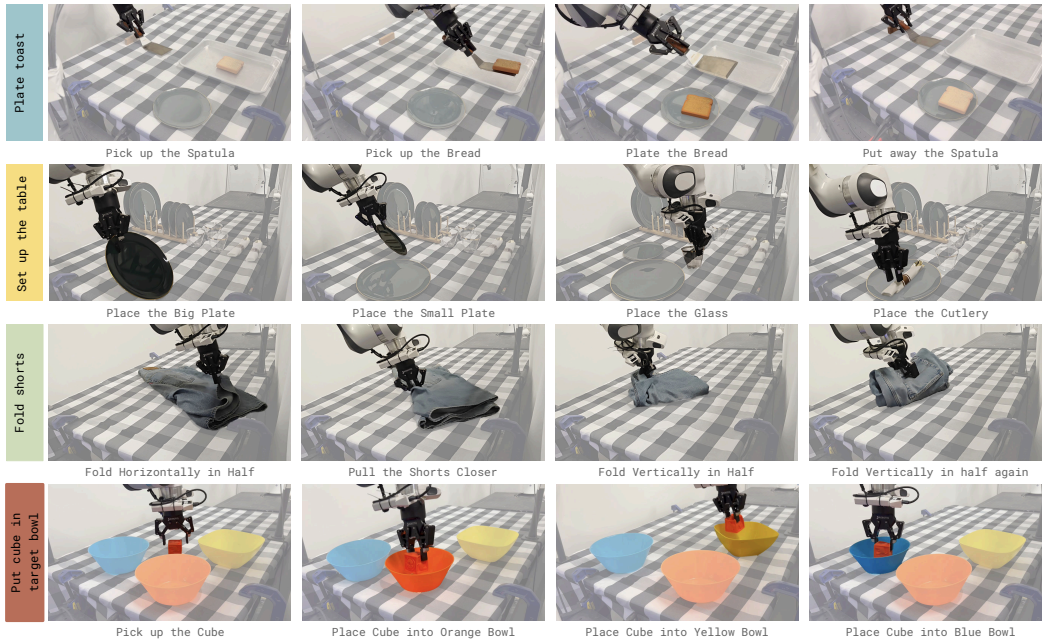


Figure 7: Real-world task suite. Each row shows key stages of the corresponding task.

Plate Toast. The robot is tasked with plating a piece of toast, either by scooping it with a spatula or picking it up directly with the gripper from a tray and placing it onto a plate. The initial position and orientation of the toast on the tray vary between episodes, as does the position of the spatula. Unlike the structured tasks, the preference axes were not fixed in advance; instead, the annotators freely specified the dimensions most relevant to them. The collected axes include speed, smoothness of motion, cleanliness, quality of plating, and whether the robot damaged the toast.

Setup Table. Here the robot lays out a full place setting on a dining table: a large plate, a small plate, a cup, and cutlery. The small plate can be placed to the left of the main plate (most formal), on top of it (medium formal), or to the right of it (least formal). The cup is placed on the opposite side of the small plate, except when the small plate is on the main plate, in which case the cup goes to the left. The cutlery is placed on the main plate in the center. The initial placement of each item on the table varies between episodes. Preferences are collected across the placement quality of each item (large plate, small plate, cup, and cutlery), formality of the resulting setup, smoothness and carefulness of motion, speed, and damage to the environment.

Fold Shorts. The robot folds a pair of shorts lying flat on a surface by performing three successive folds, each folding the garment in half. The initial configuration of the shorts, including its

position and orientation on the surface, varies between episodes. Preferences are collected along the following axes: quality and wrinkle level of each fold (1st, 2nd, 3rd), alignment of the final folded garment, speed of execution, smoothness of motion, and any damage caused to the environment.

Place Cube into Bowl. In this task, the robot places a cube in one of three bowls (blue, orange, or yellow) on the table. The initial position of the cube and the bowls varies between episodes. The task is designed to evaluate the steerability of the learned policy at test time. Preferences are collected along the following axes: target bowl placement (blue, orange, yellow), speed of execution, and smoothness of motion.

Dataset Statistics. Table 2 details the dataset statistics for real-world task.

Table 2: Dataset statistics for each task, including the number of offline demonstrations, total preference pairs, and total rollouts used for preference learning.

Task	Offline Dataset	Num Preferences	Num Rollouts	Preference axes	Num Preferences (single matching baseline only)
Plate Toast	148	295	128	5	1492
Setup Table	100	325	270	5	1625
Fold Shorts	138	239	130	10	1336*
Place Cube into Bowl	0	250	400	3	800

(*) to match the number of labels would have been prohibitively costly, we still allocate more time to collect those labels than for *FPL* for a fair comparison see Section 8 on more analysis on the cost to collect single preferences against freeform preferences.

7.2 Simulation Tasks

We propose two simulation tasks and one additional variation (see Fig. 8), which are extensions from the robomimic original tasks [53, 57]. Note that the simulation benchmarks will be available with the open-sourced version of the code. We describe them in further details below:

Object Rearrangement. This task is a modification of the pick and place task from Zhu et al. [57]. Two objects are present in the scene, the *bread* and the *can*. The goal is to place them in the correct box, in the correct order (first *bread* then *can*), and without dropping them. The offline dataset contains a mix of behaviors, data that places in the wrong order or that drops the objects as well as perfect demonstrations.

Bimodal Square. This task is a modification of the square task from Zhu et al. [57]. The goal of this task is to evaluate the compositionality of the policies. The task is considered successful if the nut is placed on the right peg, and we also keep track of the time to success. The composition of the offline dataset as shown in 4 is made of fast and slow trajectories for placing the nut on the left peg, but *only* slow trajectories for placing the nut on the right peg.

Bimodal Square (inverted). This is an extension of the *Bimodal square task*. The goal of this task is to evaluate the test-time steerability of the policies. The goal becomes to place the nut on the left peg instead of the right peg (as the success specified at training time). It is possible to steer the policy at test time for *FPL* as show in Section 5.2 but not possible for the rest of the policies.

Dataset Statistics. In Table 3, we provide further details on the dataset and preference axes for the simulation tasks.



Figure 8: Simulation task suite. Each row shows key stages of the corresponding task.

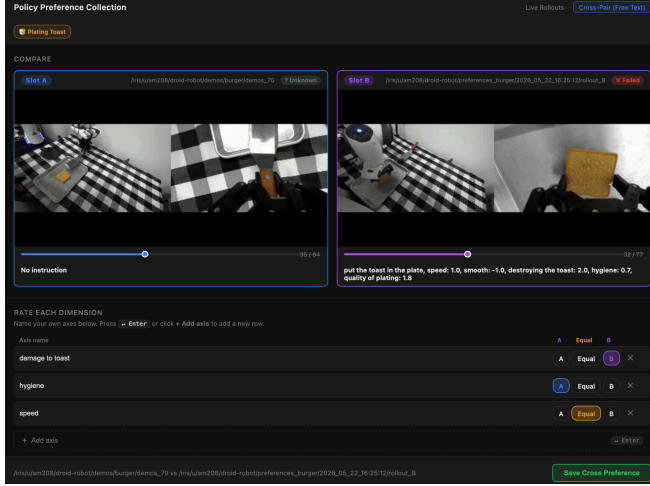
Table 3: Dataset statistics for each task, including the number of offline demonstrations, total preference pairs.

Task	Offline Dataset	Num Preferences	Preference axes	Num Preferences (single matching baseline only)
Object rearrangement	300	70	order, bread placed, can placed bread dropped, can dropped	350
Bimodal square	200	100	speed, peg	200

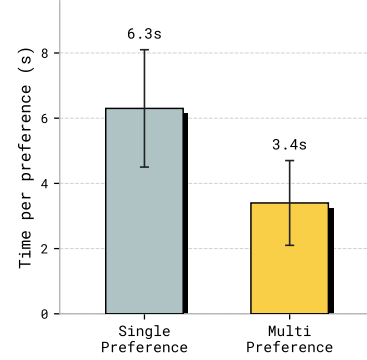
8 Preference Collection

Preferences are collected through a custom web interface in which annotators are presented with two side-by-side video recordings of robot rollouts and asked to indicate which rollout is preferred along each axis or indicate equivalence. For some tasks (Setup Table, Fold Shorts, and Place Cube into Bowl), the axes are predefined and vary by task. For the others, like the Plate Toast task, annotators additionally specify their own preference axes in free-form text prior to providing their preferences; this allows for richer feedback along dimensions of quality that would otherwise go unspecified. Figure 9a shows the data collection interface.

In Figure 9b, we show how collecting freeform preferences is cheaper in time than collecting single preferences when looking at the time spent per label. The annotator can provide multiple labels instead of only one when watching a single pair of videos of two trajectories, while for single preferences they can only provide one label, therefore the overhead cost is less compensated. The speedup observed in our experiments is of around 1.85 times faster to provide freeform preferences than single preferences.



(a) Data collection interface



(b) Annotation time

Figure 9: **(a) Data collection interface.** Annotators are shown two side-by-side robot rollout videos and asked to indicate which rollout is preferred along each axis or indicate equivalence. For open-ended tasks, annotators first specify their own preference axes in free-form text. **(b) Annotation cost.** Average wall-clock time to collect a single preference label. Under single-preference annotation, each pair of rollout videos yields one labeled axis, whereas multi-preference annotation elicits several axes from the same pair. Error bars denote one standard deviation across annotation sessions.

FPL provides flexibility in the axes of preferences. This flexibility becomes useful as the iterations of the algorithm advance and the policy performance improves. In Figure 10 which shows the per-iteration label distribution on the iterative *fold shorts* task, we observe that early iterations focus on whether each fold happens at all, and later ones, once folding is reliable, whether there are wrinkles and final alignment. Because the annotators write their own axes instead of scoring a fixed rubric, the criteria change on their own as the policy gets better, and each round ends up targeting whatever the policy is currently getting wrong, making the preferences by *FPL* more targeted.

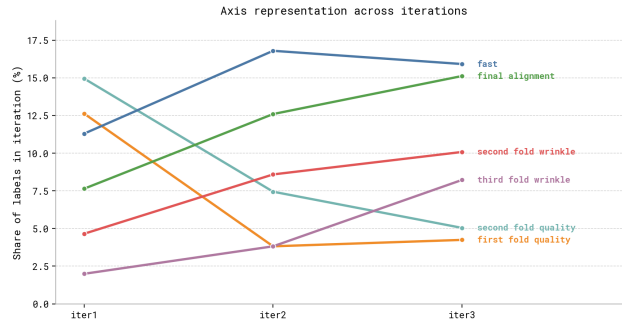


Figure 10: Share of freeform preference labels assigned to each criterion across iterations on the *fold shorts* task. As the policy improves, annotator attention shifts away from coarse fold-quality axes toward finer wrinkle and final-alignment criteria; the evaluation changes naturally without any manual redefinition of the reward.

9 Implementation Details

In this section we give further implementation details for *FPL*.

9.1 Reward-Model Learning

In order to leverage the pretraining of VLMs, we use the pretrained Qwen 3.5 VL 4B [58] model as the backbone for our real-world reward model. As presented in Fig. 2, the reward model takes

as input the question “What is the score for l_k ?” where l_k gets substituted in natural language for the name of the preference axis provided in the freeform preferences. After the prompt, we pass the images to the VLM, both first-person and third-person, for the whole sequence. Instead of passing the complete trajectory as input we stride the sequence. We freeze the vision encoder, and we full fine-tune the rest for more efficient training. We maintain a causal mask so all tokens can attend to the text and the past images, but not to the future images. We do a single forward pass across the whole sequence, and we take the final vision separator token for the second image of the pair per timestep, decode it with a single linear layer projection into a score. The final score is computed as the sum of scores across all timestep see Eq 4. When choosing the reward model, we pick the one with the highest validation accuracy in the first 500 epochs. See below for further details:

Table 4: Hyperparameters for the reward model training

Task	Sequence Length	Stride Length	Batch Size	LR	Img Size
Plate Toast	20	20	32	1e-5	128x128x3
Setup Table	20	20	32	1e-5	128x128x3
Fold Shorts	20	60	32	1e-5	128x128x3
Put cube in target bowl	20	10	32	1e-5	128x128x3

Another detail is that we allow annotators to specify that two trajectories are equivalent in some axis, in those cases we drop the label.

9.2 Policy Learning

Reward-Conditioning. In order to determine the reward conditioning, we run inference on the whole dataset $\mathcal{D}$ with the reward model on the selected axes L . We compute the statistics of the reward and obtain mean μ and standard deviation σ . Then, we standardize the reward $r_{cond} = \frac{r - \mu}{\sigma}$. This makes the rewards bounded and easy to specify at test-time without additional search. The reward conditioning is passed to the policy π for trajectory τ through text by passing l_k : $round\left(\frac{r_{\phi}(l_k, \tau) - \mu}{\sigma}\right)$ where we pass the scores round up to 1 decimal place.

We use the $\pi_{0.5}$ [59] open source weights as the VLA pretraining, which uses a flow matching action head with action chunk size 16 and rollout 8 actions. We do full finetuning of the policies over 30000 steps with a batch size of 32. The policy obtains the wrist and the third person camera views as input as well as the task prompt with the conditioning as through text described above. Thereafter an example prompt for the VLA will be the following:

Fold Shorts

fold the shorts, Quality of 1st fold: 2.2, Wrinkle of 1st fold: 2.1, Quality of 2nd fold: 0.7, Wrinkle of 2nd fold: 0.8, Quality of 3rd fold: -0.5, Wrinkle of 3rd fold: -0.5, Alignment of final fold: 0.1, Fast: 1.2, Smooth: 1.2, Damage to environment: -0.1

Plate Toast

put the toast in the plate, speed: 0.5, smooth: -0.8, destroying the toast: -1.5, hygiene: -2.1, quality of plating: -0.2

Setup Table

set up the table, Quality of placement of big plate: -0.8, Quality of placement of small plate: -0.1, Quality of placement of cup: 0.1, Quality of placement of cutlery: -0.9, Formality of setup: -0.1

Put cube in target bowl

put the cube in the bowl, Blue bowl: -0.5, Orange bowl: 2.0, Yellow bowl: -0.3

9.3 Simulation setup

In simulation, we prioritize speed of iteration, thereafter the environments and policies work directly from the state space. In that case, we don’t use any of the pretrained models and train everything from scratch. For the reward model, instead of taking language as command, we have K different heads, one per preference axis, and we predict all of the axes at the same time. The policy remains a flow-matching policy based on the diffusion transformer code from Chi et al. [25]. Instead of taking the conditioning as input through text, we encode the input as a float vector and pass it to the policy directly. We observed that for the simulation experiments not to overfit to this input conditioning vector, we had to add some noise of 0.2 after standardization of the rewards. This makes it more robust; however, note that this is not necessary in the real world and is only needed here because we train a smaller network from scratch, which is more prone to overfitting.

9.4 Baselines

We compare *FPL* against six baselines that ablate key components of our method.

Behaviour Cloning (BC) trains a policy via supervised imitation learning on all offline demonstrations $\mathcal{D}$. Preference data are not used, and the policy does not receive reward conditioning.

Filtered BC extends vanilla BC by augmenting the offline dataset $\mathcal{D}$ with rollouts from the preference dataset that were labeled successful. As with BC, no reward conditioning is used; this baseline isolates the benefit of additional on-policy data from the benefit of preference supervision.

Single Preference with matching pairs follows the same reward-conditioned policy extraction as *FPL* (Eq. 5), but replaces the multi-dimensional reward with a single scalar reward trained on a user-specified “overall quality” axis. The reward model is trained via the Bradley–Terry objective (Eq. 2) on a single axis of feedback. This baseline isolates the benefit of multi-axis feedback from the benefit of reward conditioning alone. This baseline uses as many pairs of trajectories compared as *FPL*.

Single Preference with matching comparisons is exactly the same as the *Single Preference* baseline described above, but instead of matching the number of pairs compared, we match the number of comparisons that the human provides. For *Single Preference* we train with one label per comparison, while for *FPL*, we train on K labels per comparison. To demonstrate that the benefit of *FPL* comes from the detail of the preferences and not from a difference in the amount of labels we collect K times more comparisons for this baseline in order to have a fairer comparison. See Table 2 and 3, where we explicitly show the number of labels and comparisons collected for this baseline versus the rest.

Advantage Conditioning is based on Intelligence et al. [20]. Instead of learning the reward model from preferences, we learn from success/failure rewards where the reward at each timestep t is:

$$r_t = \begin{cases} 0 & \text{if } t = T \text{ and trajectory succeeded} \\ R_{\text{fail}} & \text{if } t = T \text{ and trajectory failed} \\ -1 & \text{otherwise} \end{cases} \quad (6)$$

where $R_{\text{fail}} = -100$.

The value target is the Bellman backup with no discount; we normalize them so the magnitude does not hurt the fit.

$$V(s_t) = \sum_{k \geq t} r_k \quad (7)$$

We then condition the policy on the advantage

$$A_t = V(s_{t+1}) - V(s_t) \quad (8)$$

Which is normalized in the same way as *FPL* when passed as input to the policy, for easily finding the correct values to condition at test time. The policy learning part follows the same recipe as *FPL*.

Weighted Regression We train the reward model on multi-dimensional preferences using the same recipe as for *FPL*. The difference is that instead of conditioning the policy on the reward scores, we use a different policy extraction method based on Advantage Weighted Regression [54]. We average all the scores obtained from the reward model over the L selected axes, and we modify the loss by weighting it by the average of the scores:

$$\mathcal{L}_{\text{AWR}}(\theta) = - \frac{\mathbb{E}_{\tau_i \sim \mathcal{D}} \left[\sum_{t=1}^{T_i} \log \pi_{\theta}(\mathbf{a}_i^t \mid s_i^t) \exp \left(\sum_k^{K_{\pi}} r_{\phi}(\tau_i \mid l_k) \right) \right]}{\sum_{j=0}^{|\mathcal{D}|} \sum_k^{K_{\pi}} r_{\phi}(\tau_j \mid l_k)}, \quad (9)$$

We refer to Table 8 and Table 5 for all the detailed numbers on the real-world and simulation benchmarks.

10 Compute details

The compute resources used for this project were one H100 for full finetuning of the policy and finetuning of the reward model. For the simulation experiments, smaller GPUs were used, generally NVIDIA RTX 4500.

11 Detailed Results

11.1 Simulation Tasks

Table 5 reports the performance of each one of the baselines in simulation.

Table 5: Simulation results comparing *FPL* against baselines across simulation environments. Multi-dimensional preferences as leveraged by *FPL* provide the best supervision.

Method	Object rearrangement (success)	Bimodal square (throughput)	Bimodal square inverted (throughput)
BC [55]	0.04 ± 0.02	0.71 ± 0.07	0.25 ± 0.14
Filtered BC [29]	0.09 ± 0.05	0.71 ± 0.02	0.00 ± 0.00
Weighted Regression [54]	0.18 ± 0.08	0.83 ± 0.08	0.09 ± 0.06
Advantage Conditioning [20]	0.17 ± 0.06	0.75 ± 0.12	0.08 ± 0.04
Single Preferences [7]	0.73 ± 0.04	0.67 ± 0.08	0.78 ± 0.01
Single Preferences (matching labels)	0.79 ± 0.06	0.73 ± 0.05	0.74 ± 0.05
<i>FPL</i> (Ours)	0.84 ± 0.09	1.19 ± 0.01	1.24 ± 0.01

Tables 6 and 7 report the detailed per-metric breakdown for the Bimodal Square environment in the standard and inverted conditioning directions, respectively, including success rate, time to first step (lower is better), and throughput.

Table 6: Detailed results on the Bimodal Square environment (standard direction). We report mean ± standard error over 3 seeds for success rate, time to first step (TIS, lower is better), and throughput.

Method	Success	TIS	Throughput
Demo only	0.73 ± 0.15	347.71 ± 7.14	0.63 ± 0.13
Filtered BC	0.92 ± 0.03	405.68 ± 3.36	0.68 ± 0.02
Weighted Reg.	0.90 ± 0.07	333.77 ± 20.45	0.81 ± 0.10
Single Pref	0.89 ± 0.11	395.93 ± 1.80	0.67 ± 0.08
Single Pref. Matching Labels	0.95 ± 0.05	391.80 ± 6.10	0.73 ± 0.05
Advantage Cond.	0.89 ± 0.06	356.31 ± 31.81	0.75 ± 0.12
<i>FPL</i> (Ours)	1.00 ± 0.00	253.13 ± 0.01	1.19 ± 0.00

Table 7: Detailed results on the Bimodal Square environment (inverted direction). We report mean $\pm$ standard error over 3 seeds for success rate, time to first step (TIS, lower is better), and throughput.

Method	Success	TIS	Throughput
Demo only	0.27 ± 0.15	330.12 ± 12.37	0.25 ± 0.14
Filtered BC	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Weighted Reg.	0.09 ± 0.08	292.26 ± 37.31	0.09 ± 0.06
Advantage Cond.	0.09 ± 0.06	317.99 ± 60.07	0.08 ± 0.04
Single Pref. Matching Labels	0.96 ± 0.02	390.67 ± 18.22	0.74 ± 0.05
Single Pref	0.97 ± 0.01	376.50 ± 4.96	0.78 ± 0.01
<i>FPL</i> (Ours)	1.00 ± 0.00	242.23 ± 0.06	1.24 ± 0.00

11.2 Real World Tasks

Table 8 reports per-task success rates for real-world experiments across all methods.

Table 8: Success rate per task and method. We report the mean $\pm$ standard error.

Method	Setup Table	Plate Toast	Fold Shorts	Place Cube into Bowl	Average
BC	0.68 ± 0.05	0.15 ± 0.08	0.10 ± 0.07	0.33 ± 0.10	0.31 ± 0.04
Filtered BC	0.65 ± 0.04	0.30 ± 0.10	0.25 ± 0.10	0.29 ± 0.10	0.37 ± 0.04
Single Preference (match pairs)	0.64 ± 0.06	0.00 ± 0.00	0.25 ± 0.10	0.43 ± 0.11	0.33 ± 0.04
Single Preference (match comparisons)	0.77 ± 0.03	0.10 ± 0.07	0.30 ± 0.10	0.19 ± 0.09	0.34 ± 0.04
<i>FPL</i> (Ours)	0.94 ± 0.02	0.70 ± 0.10	0.55 ± 0.11	0.81 ± 0.09	0.75 ± 0.04

11.2.1 Steerability of the policies

In Figure 5, we depict the results from Table 8 for the place cube into bowl task. More concretely, we show an overlay of the commanded bowl and the bowl where it was placed. We observe that *FPL* can achieve steerability by changing at test-time which reward dimension we want the policy to optimize, while the baselines do not.

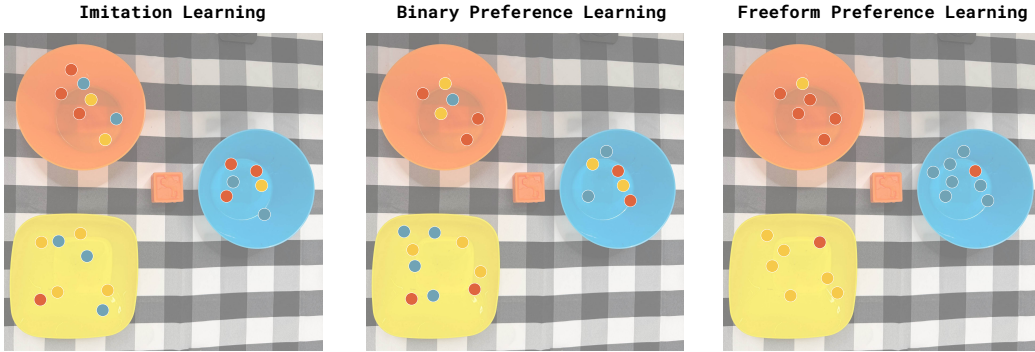
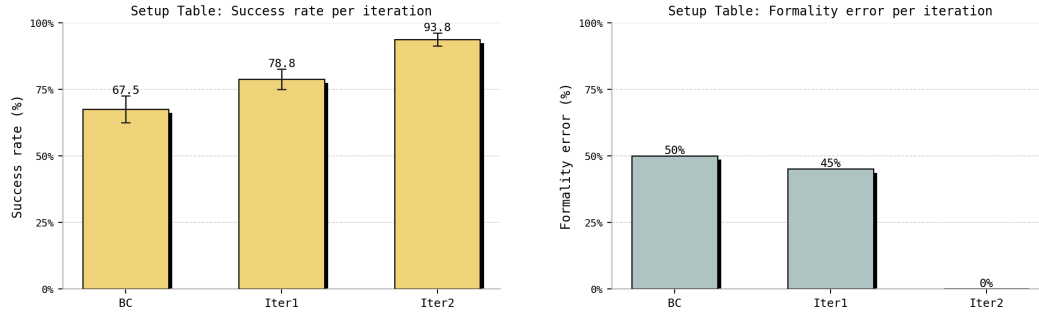


Figure 11: Steerability Test. Each dot represents the block’s final position, with the color corresponding to the target bowl’s color. *FPL* achieves steerability by placing the cube in the commanded bowl most of the time, whereas the placement is close to random for the other two baselines. Steerability is achieved for *FPL* by prompting, at test time, the reward it needs to optimize.

11.2.2 Iterative Improvement

Figure 12 shows how *FPL* improves policy between iterations of the Setup Table task, compared to BC. We report the task success rate (higher is better) and the formality error rate (lower is better).



(a) Policy success rate per iteration on the *setup table* task.

(b) Formality error rate.

Figure 12: Iterative analysis on the *setup table* task: (a) policy success climbs across iterations, and (b) the formality error rate falls to 0.

Figure 13 shows the analogous progression on the iterative Fold Shorts task, where preferences are collected in *freeform* format. Policy success increases across iterations relative to the BC baseline.

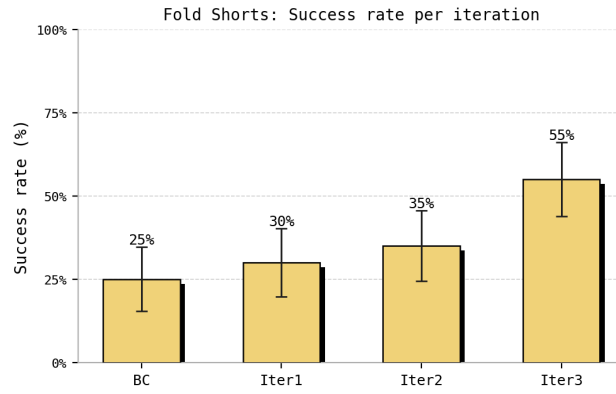


Figure 13: Policy success rate per iteration on the *fold shorts* task.

Preference-criterion vocabulary (burger / plating-toast)

41 distinct criterion labels grouped into 6 topics · 1440 total mentions · tile size = frequency within topic · darker shade = more frequent synonym



Figure 14: Freeform preference labels collected for the *plate toast* task grouped by topic. Tile size and shade reflect label frequency.